西南科技大学本科毕业论文（设计）开题报告

学　院	计算机科学与技术学院	**专业**	软件工程	**班级**	软件 2301
姓　名	宋文頔	**学号**	5120232314	**指导教师**	苏波
论文（设计）题目	基于大语言模型的 Java 代码智能审查与缺陷定位系统的设计与实现				

一、选题背景（目的、意义）

随着软件系统规模持续扩大和迭代节奏不断加快，代码评审已成为保障软件质量、控制变更风险的关键环节。在 Java 企业级项目中，一次合并请求往往涉及多个模块和数百行代码，人工评审不仅耗时费力，而且高度依赖评审者的经验与状态，容易出现漏看、标准不一致等问题；对于中小团队而言，专职评审资源更加有限，代码质量问题常常在测试甚至上线后才暴露，修复成本显著上升。

为降低人工负担，SonarQube、SpotBugs、PMD 等静态分析工具被广泛用于代码检查。这类工具基于预定义规则和模式匹配，能够快速发现空指针、资源泄漏、并发安全等典型问题，但其判断依据是固定规则，缺乏对代码语义、调用上下文和业务意图的理解，往往产生大量误报，开发者需要逐条人工甄别，反而增加了审查负担。近年来，以 CodeBERT、Codex、Qwen2.5-Coder 为代表的大语言模型在代码理解、缺陷解释和修复建议生成方面表现出较强能力，为智能代码审查提供了新思路。然而，大语言模型受上下文窗口限制，难以直接处理大型工程；若仅输入变更片段，又会丢失跨文件调用关系和项目规范等关键信息，导致判断片面；同时模型可能产生看似合理但实际错误的“幻觉”，直接用于审查难以保证结果的准确性与可解释性[1][2]。

针对上述问题，本课题将程序静态分析与大语言模型相结合，设计并实现面向 Java 项目的智能代码审查与缺陷定位系统。系统以抽象语法树、控制流图、数据流图和调用关系为基础，对变更代码进行结构化解析与切片，为模型提供精准、可追溯的上下文；利用检索增强生成技术从项目代码库、历史审查记录和编码规范中检索相似片段与依据，弥补模型的项目级知识不足；再将静态规则检测结果与大模型语义判断进行融合，通过置信度评分和交叉验证抑制误报。其意义主要体现在三个方面：一是提升审查准确性，通过精准切片减少无关信息干扰，并以规则与模型的双重校验降低误报；二是提高审查效率与可解释性，系统接入代码托管平台的合并请求流程，自动定位缺陷所在文件与行号，给出缺陷类型、风险等级、成因说明和修复建议，使审查结果可直接用于开发实践；三是具备良好的实用与推广价值，系统采用插件化规则和多模型接入方式，可适配不同规模的代码库和大模型服务，为中小团队提供低成本的代码质量保障手段[1][3][5]。

二、国内外研究现状综述

（一） 国外研究进展

国外对代码智能分析与评审自动化的研究起步较早。在缺陷检测方面，Allamanis 等系统梳理了机器学习用于“大代码”的研究脉络，指出代码具有自然性与规律性，可通过统计学习捕获潜在缺陷模式[7]；CodeBERT、GraphCodeBERT 等预训练模型将代码的文本与结构信息联合建模，显著提升了代码分类、克隆检测和缺陷预测等任务的效果[15]。在代码评审自动化方面，Tufano 等对现有自动化评审技术进行了系统比较，指出基于预训练模型的方法在评论生成和代码修改建议上已取得进展，但在复杂语义理解和跨文件上下文方面仍存在明显不足[3]；Li 等提出 CodeReviewer，通过大规模代码变更与评审语料预训练，将评审任务统一为评论生成和代码精炼，验证了预训练在评审场景中的有效性[4]。Hou 等对 2020 年以来大语言模型在软件工程中的应用进行了系统性文献综述，覆盖需求、代码生成、测试、评审与维护等任务，指出大模型在代码评审中的潜力与幻觉、可解释性等挑战并存[2]。

在检索增强与仓库级理解方面，Lewis 等提出的 RAG 框架将外部知识检索与生成模型结合，为知识密集型任务提供了可溯源的答案[8]；RepoCoder 进一步将迭代检索—生成机制用于仓库级代码补全，证明引入项目内相似代码能够有效弥补模型上下文不足[9]。上述工作为“以检索补全项目上下文”的审查思路提供了直接依据。在静态分析方面，FindBugs/SpotBugs 等工具通过字节码模式匹配检测缺陷，Ayewah 等对其误报与使用情况进行了实证分析，指出规则型工具的高误报是限制其落地的主要因素[14]；SonarQube 则在工程实践中提供了较完整的规则体系与质量门禁。总体来看，国外研究已形成“预训练模型+静态分析+检索增强”的多条技术路线，但将三者深度协同并面向合并请求开展增量审查的系统性工作仍较少。

（二） 国内研究进展与工程实践

国内在代码审查自动化与大模型辅助软件工程方面进展迅速。花子涵等对代码审查自动化研究进行了系统综述，从规则与静态分析、基于语料库、基于深度学习等角度归纳了现有方法，指出评审意见生成、缺陷定位和误报控制是当前研究的重点与难点[1]。蒋远等针对基于大语言模型的源代码漏洞检测进行了综述，从轻量级 Transformer 与大模型两条路线梳理了提示工程、指令微调、混合分析等技术，认为大模型擅长发现静态规则难以刻画的复杂逻辑缺陷，但受数据质量、上下文长度和可解释性制约[5]。刘天阳等以 Java 程序资源泄漏为对象，对比了传统静态模型与语言模型的有效性，发现二者在召回与误报上具有互补性，为规则与模型融合提供了实证支持[6]。

在工程实践方面，Qwen2.5-Coder、DeepSeek-Coder 等国产代码模型在多项代码理解与生成基准上取得较好成绩，并支持本地化部署，为中文项目和多模型接入提供了条件[16]；Gitee/GitCode、阿里云效 Codeup、华为云 CodeArts 等代码托管与研发平台也开始集成 AI 评审、代码扫描和合并请求检查能力。但现有平台多以通用规则或单轮问答为主，审查结果与项目上下文、历史评审记录的关联不足，缺陷定位粒度和修复建议的可执行性仍有提升空间。

（三） 关键方法与工具最新进展

在程序分析方面，JavaParser 和 Spoon 是使用最广泛的 Java 源码解析与转换库。JavaParser 提供完整的 AST 构建、符号解析与访问者机制，适合提取方法、类型、调用关系等结构化信息[12]；Spoon 支持更细粒度的模型构建与代码变换，便于实现自定义分析[11]。程序切片技术由 Weiser 提出，通过数据流和控制流依赖提取与目标语句相关的程序子集，为聚焦审查范围、减少模型输入噪声提供了理论基础[10]。在检索与存储方面，pgvector、Milvus 等向量数据库支持大规模向量相似度检索，LangChain4j、Spring AI 等框架提供模型调用、提示模板和检索链的工程封装，使 RAG 流程能够快速落地。在评测方面，Defects4J 收录了大量真实 Java 项目缺陷及触发测试，为缺陷检测与定位方法的可复现对比提供了基准[13]。这些成熟工具与基准显著降低了本课题的工程实现与实验验证难度。

（四） 综述性结论：研究空白与本课题切入点

综合国内外研究可以看出：第一，规则型静态分析工具虽然高效，但缺乏语义理解，误报率较高；纯大模型方案虽具备语义推理能力，却受上下文窗口和幻觉制约，二者尚未形成稳定、可解释的深度融合机制[1][14]。第二，现有代码评审研究多聚焦评论生成或单文件缺陷预测，对跨文件调用、项目规范和业务约束等仓库级上下文的构造仍不充分，检索增强在代码审查中的应用多停留在相似代码召回层面[2][8][9]。第三，面向合并请求的增量审查、缺陷精确定位、风险分级和修复建议生成缺乏端到端的工程化系统，且缺少在真实 Java 缺陷集上的统一评测。第四，中小团队对低成本、可本地化部署的审查工具需求迫切，但现有商业平台价格较高、规则封闭。基于此，本课题拟以“结构化切片提供精准上下文、检索增强补全项目知识、规则与模型融合抑制误报、合并请求集成实现增量审查”为切入点，构建可评测、可扩展的 Java 代码智能审查与缺陷定位系统。

三、研究目标与研究内容

（一）研究目标

本课题旨在设计并实现一个基于大语言模型的 Java 代码智能审查与缺陷定位系统，将程序静态分析、检索增强生成与大模型语义推理相结合，在保证审查结果可解释、可追溯的前提下，提高缺陷发现的准确率与审查效率。具体目标包括：（1）实现 Java 源码解析、调用关系构建与增量差异分析，形成面向审查任务的代码切片方法；（2）设计检索增强的上下文构造策略与提示工程方案，提升模型对项目级上下文的理解能力；（3）构建静态规则与大模型协同的缺陷判定及误报抑制机制，覆盖空指针、资源泄漏、并发安全、异常处理等典型缺陷；（4）实现与代码托管平台的集成、审查报告可视化及结果反馈功能；（5）构建缺陷标注评测集，以准确率、召回率、F1 值、误报率和响应时延等指标开展对比实验与验证。

（二）研究内容

围绕上述目标，本课题的研究内容包括需求分析、总体架构与数据库设计、各功能模块

实现以及实验验证四个层面。系统采用前后端分离架构，后端基于 Spring Boot 3 与 LangChain4j/Spring AI，数据库采用 MySQL 与 pgvector，缓存采用 Redis，前端采用 Vue 3，并通过 Docker 完成部署；大模型经 OpenAI 兼容接口调用云端服务，也可选配 Ollama 运行轻量本地模型，保证在无 GPU 的普通计算机上完成开发与演示。

具体实现以下七个模块：（1）代码仓库接入模块：支持 Git 仓库克隆、分支管理与合并请求差异解析，提取变更文件、变更类型及代码行范围，为增量审查提供输入。（2）代码解析与切片模块：基于 JavaParser/Spoon 构建抽象语法树，提取方法级代码、调用链、类型信息与上下文依赖，结合控制流与数据流分析生成面向缺陷的代码切片。（3）检索增强模块：对项目代码、历史审查记录和编码规范建立向量索引，检索相似代码片段与规范条款，为模型提供项目级上下文和判断依据。（4）智能审查模块：设计缺陷检测提示模板与多轮推理流程，实现缺陷识别、严重程度分级、文件与行号定位、成因解释和修复建议生成，支持多模型接入与结果缓存。（5）误报抑制模块：融合 SonarQube、SpotBugs 等静态分析规则与大模型判断，设计置信度评分和交叉验证机制，对低置信度告警进行复核，降低误报率。（6）报告与平台集成模块：生成可视化审查报告，通过 Webhook 将结果回写至代码托管平台，提供历史记录查询、规则配置和模型切换功能。（7）测试与评估模块：编写单元测试和集成测试，构建缺陷标注数据集，对比基线方法在准确率、召回率、F1 值和误报率上的表现，并测试系统在不同规模项目下的响应时延与稳定性。

四、拟采用的研究思路（方法、技术路线、可行性论证等）

（一）研究思路

本课题采用“静态分析筑基、检索增强补上下文、大模型语义审查、规则融合抑制误报”的四线协同思路。首先利用程序分析技术将源代码转换为结构化表示，明确变更影响范围与依赖关系；其次以代码切片和向量检索为模型提供精准、可追溯的上下文，降低无关信息与上下文长度的影响；然后通过精心设计的提示模板引导大模型完成缺陷识别、定位、解释与修复建议生成；最后将静态规则结果与模型判断进行交叉验证和置信度融合，形成可解释的审查结论。研究过程遵循“方法验证—系统集成—量化评测”的递进路线，先在小规模样例上验证切片与提示策略的有效性，再完成系统集成，最后在真实缺陷集上开展对比实验。

（二）技术路线

技术路线可分为五个阶段。第一阶段为数据与仓库接入：完成 Git 仓库克隆、分支管理和合并请求差异解析，提取变更文件及行号范围。第二阶段为代码解析与切片：使用 JavaParser/Spoon 构建 AST，结合符号解析生成方法级代码、调用链、类型信息与控制流/数据流依赖，形成面向审查任务的代码切片。第三阶段为检索增强：对代码片段、历史审查记录和编码规范进行向量化并存入 pgvector，审查时检索相似片段与规范条款，与切片共同构成模型输入。第四阶段为智能审查与误报抑制：设计缺陷检测提示模板，采用多轮推理输出结构化的缺陷类型、严重程度、位置、成因和修复建议；同时调用 SonarQube/SpotBugs 获取规则告警，通过置信度评分和交叉验证融合两类结果。第五阶段为平台集成与评测：生成可

视化报告并通过 Webhook 回写代码托管平台，基于 Defects4J 和自建标注集，以准确率、召回率、F1 值、误报率和响应时延为指标开展对比实验。技术路线如图 4-1 所示。

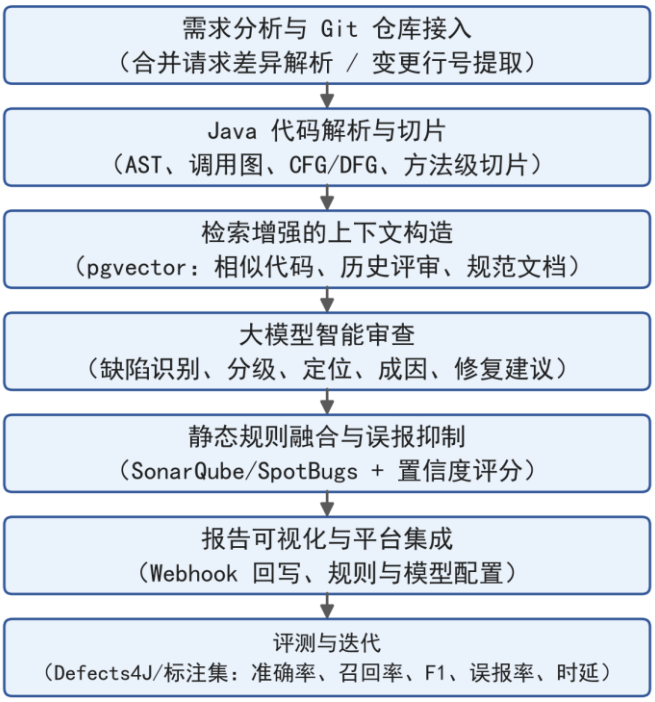


图 4-1 技术路线图

（三）可行性论证

技术层面，JavaParser、Spoon、SonarQube、SpotBugs 等工具均已成熟并有完善文档，Spring Boot 3、LangChain4j/Spring AI、pgvector、Vue 3 和 Docker 具备大量工程实践，系统所需的关键技术均有可参考的实现方案[11][12][13][14]。数据层面，Defects4J 提供了真实 Java 项目缺陷及触发测试，开源代码托管平台可获取大量 Java 项目和历史评审记录，能够支撑评测集的构建[13]。条件层面，系统通过 OpenAI 兼容接口调用云端大模型，也可使用 Ollama 运行轻量本地模型，无需高性能 GPU 即可完成开发与演示；开发环境、数据库和中间件均可通过 Docker 快速搭建。

工程层面，课题按照需求分析、总体设计、模块实现、系统集成和测试评估的软件工程流程推进，各模块职责清晰、接口明确，便于分阶段验证。进度层面，开题后 24 周内安排了文献调研、数据准备、核心模块实现、系统集成、评测实验和论文撰写等环节，时间分配合理，关键风险点均预留了缓冲。综合来看，本课题在技术、数据、软硬件条件和进度安排上均具备可行性。

五、研究工作进度安排

工作进度安排(2026 年 11 月-2027 年 5 月，合计 24 周)：

表格 5-1 工作安排进度表

时间	完成的任务
第 1-4 周	文献调研、需求分析、完成开题报告
第 5-6 周	代码仓库接入、合并请求差异解析、测试数据准备
第 7-9 周	Java 代码解析、调用图构建与代码切片
第 10-11 周	向量索引构建与检索增强模块实现
第 12-14 周	智能审查、静态规则融合与误报抑制
第 15-16 周	平台集成、可视化报告与 Webhook 回写
第 17-18 周	构建评测集，完成对比实验与性能测试
第 19-24 周	撰写论文、系统优化与答辩材料准备

六、参考文献

[1] 花子涵, 杨立, 陆俊逸, 等. 代码审查自动化研究综述[J]. 软件学报, 2024, 35(7): 3265-3290.

[2] Hou X, Zhao Y, Liu Y, et al. Large Language Models for Software Engineering: A Systematic Literature Review[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(8): 220:1-220:79.

[3] Tufano R, Dabić O, Mastropaolo A, et al. Code Review Automation: Strengths and Weaknesses of the State of the Art[J]. IEEE Transactions on Software Engineering, 2024, 50(2): 338-353.

[4] Li Z, Lu S, Guo D, et al. Automating Code Review Activities by Large-Scale Pre-training[C]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 1035-1047.

[5] 蒋远, 杨子含, 苏小红, 等. 基于大语言模型的源代码漏洞检测方法综述[J]. 计算机学报, 2026, 49(6): 1385-1423.

[6] 刘天阳, 叶嘉威, 计卫星, 等. Java 程序资源泄露缺陷检测：传统模型和语言模型的有效性分析[J]. 软件学报, 2025, 36(6): 2432-2452.

[7] Allamanis M, Barr E T, Devanbu P, et al. A Survey of Machine Learning for Big Code and Naturalness[J]. ACM Computing Surveys, 2018, 51(4): 81:1-81:37.

[8] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems 33. 2020: 9459-9474.

[9] Zhang F, Chen B, Zhang Y, et al. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 2023: 2471-2484.

[10] Weiser M. Program Slicing[J]. IEEE Transactions on Software Engineering, 1984, SE-10(4): 352-357.
[11] Pawlak R, Monperrus M, Petitprez N, et al. SPOON: A Library for Implementing Analyses and Transformations of Java Source Code[J]. Software: Practice and Experience, 2016, 46(9): 1155-1179.
[12] Smith N, van Bruggen D, Tomassetti F. JavaParser: Visited: Analyse, Transform and Generate Your Java Code Base[M]. Victoria: Leanpub, 2017.
[13] Just R, Jalali D, Ernst M D. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs[C]//Proceedings of the 2014 International Symposium on Software Testing and Analysis. 2014: 437-440.
[14] Ayewah N, Pugh W, Morgenthaler J D, et al. Using Static Analysis to Find Bugs[J]. IEEE Software, 2008, 25(5): 22-29.
[15] Feng Z, Guo D, Tang D, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages[C]//Findings of the Association for Computational Linguistics: EMNLP 2020. 2020: 1536-1547.
[16] Qwen Team. Qwen2.5-Coder Technical Report[EB/OL]. (2024-09-18)[2026-11-03]. https://arxiv.org/abs/2409.12186.

指导教师意见	指导教师（签名）_____ 年 月 日
答辩小组意见	□通过 □不通过 答辩组成员（签名）_____ 答辩组组长（签名）_____ 年 月 日

学院审核意见	分管教学院领导签字（公章）_____ 年　月　日